

0. SETUP (1ª CÉLULA)

```
import pandas as pd
postos = pd.read_csv("postos_sp_2025.csv", sep=";", decimal=",")
distrib = pd.read_csv("distribuicao_municipios_sp_2025.csv",
                      sep=";", decimal=",")

# Strip espaço fantasma em todas colunas de texto
for col in postos.select_dtypes(include="object").columns:
    postos[col] = postos[col].str.strip()

postos["Mes"] = (postos["Data da Coleta"].str[6:10] + "-"
                + postos["Data da Coleta"].str[3:5])
print(postos.shape, distrib.shape); postos.head()
```

CSV BR: sempre sep=";" + decima=",". Sem isso, parse quebra ou número vira string.

1. INSPEÇÃO

df.head(n)	tail()	sample()	linhas
df.shape (sem ())	(lin, col)		
df.info()	dtypes	columns	tipos/nomes
df.describe()	num: count/mean/std/min/Q/max		
describe(include="all")	+ texto		
len(df)	nº linhas		

2. SELEÇÃO DE COLUMNA

```
df["c"]          # Series (1D)
df[["c1","c2"]]  # DataFrame (várias, ordem da lista)
df[["c"]]        # DataFrame com 1 col (≠ Series)
```

1 colchete vira Series, 2 colchetes viram DataFrame. Pra reordenar colunas, usa 2.

3. FILTRO LÓGICO (LINHA)

```
cond = df["Produto"] == "GASOLINA"    # Series booleana
df[cond]                               # filtra

df[df["Produto"] == "GASOLINA"]
df[df["Valor de Venda"] > 6]
df[df["Bandeira"] != "BRANCA"]
df[df["Bandeira"].isin(["VIBRA","RAIZEN"])]
len(df[df["Produto"]=="GASOLINA"])    # quantas
```

Ops: == != > < >= <= e .isin([...]).

4. AND / OR / NOT

```
# NUNCA and/or/not. Sempre & | ~
df[(df["Produto"]=="GASOLINA") & (df["Valor de Venda"]>6)]
df[(df["Bandeira"]=="VIBRA") | (df["Bandeira"]=="RAIZEN")]
df[~(df["Bandeira"]=="BRANCA")]

# Recomendado (cada cond como variável):
c1 = df["Produto"]=="GASOLINA"
c2 = df["Valor de Venda"]>6
df[c1 & c2]
```

Cada condição entre parênteses. Esquecer é erro #1.

5. LOC / ILOC

```
df.loc[cond, "Valor de Venda"]
df.loc[cond, ["Revenda","Valor de Venda"]]
df.loc[df["Produto"]=="GASOLINA","Valor de Venda"].mean()
df.iloc[0]          # 1ª linha (posição)
df.iloc[0:5, 0:3]   # FIM EXCLUSIVO
df.loc[5:10]        # FIM INCLUSIVO
```

	loc	iloc
Usa	rótulo	posição
Fim	inclusivo	exclusivo

6. .STR (STRINGS NA COLUMNA)

```
df["c"].str.upper() / .lower() / .title() / .strip()
df["c"].str.len()          # tamanho
df["c"].str.contains("POST0") # booleana (filtro!)
df["c"].str.startswith("AUTO") / .endswith("LTDA")
df["c"].str.replace("LTDA","") # SUBSTRING
df["c"].str[0] .str[-3:] .str[6:10] # slice
df["c"].str.split()        # vira lista
df["c"].str.split(expand=True) # vira DF
df["c"].str.split().str[-1] # último elemento

df["c"].str.upper().str.replace("LTDA","").str.strip() # encadeia
```

Sem .str: 'Series' object has no 'upper'. Sempre .str antes.

7. REPLACE VS .STR.REPLACE

Forma	Compara
col.replace("SP","BA")	valor inteiro
col.str.replace("SP","BA")	subtring no texto

```
df["Estado"] = df["Estado"].replace("SP","Sao Paulo")
df["Bandeira"] =
df["Bandeira"].replace({"BRANCA":"SEM","VIBRA":"BR"})
```

8. CRIAR / ATUALIZAR COLUMNA

```
df["nova"] = df["a"] + df["b"] # cria à direita
df["a"] = df["a"] * 1.1        # substitui
df["Pais"] = "Brasil"          # constante
df["Cid_U"] = df["Município"].str.upper()
```

Pegadinha mortal: df = df["c"].str.lower() DESTRÓI o DF (vira Series). Esquerda tem que ser df["c"], NUNCA df.

9. ASTYPE (CONVERTER TIPO)

```
df["c"] = df["c"].astype(float / str / int)

# Se Valor veio string (decimal não pegou):
df["Valor de Venda"] = (df["Valor de Venda"]
                        .str.replace(",",".").astype(float))
```

String + número falha. Converte: "C" + df["id"].astype(str).

10. ESTATÍSTICA NUMÉRICA

```
df["c"].mean() .median() .std() .var()
df["c"].min() .max() .sum() .count() # count = não-nulos
df["c"].quantile(0.25) .quantile(0.75)
df["c"].describe() # tudo de uma vez
df["c"].idxmax()   # ÍNDICE do maior
```

Sem (): df["c"].mean retorna bound method, não o número.

11. VALUE_COUNTS / UNIQUE / MODE

```
df["c"].value_counts() # contagem decrescente
df["c"].value_counts(normalize=True) # proporção
df["c"].value_counts(dropna=False) # inclui NaN
df["c"].value_counts().head(3) # top 3
df["c"].value_counts()["VIBRA"] # 1 valor

df["c"].unique() .nunique() .mode()
```

Que valores?	unique
Quantos distintos?	nunique
Quantos de cada?	value_counts
Mais frequente?	mode

12. ORDENAÇÃO / DUPLICATAS / NULOS

```
df.sort_values("c", ascending=False) # NÃO persiste sozinho
df = df.sort_values("c")              # PERSISTE
df.drop_duplicates(subset=["CNPJ"])

df.isnull().sum() # nulos por coluna
df.dropna(subset=["c"]) # remove linhas com NaN em c
df.fillna(0) # preenche
```

13. GROUPBY (OPERAÇÃO-CHAVE)

```
# df.groupby(CRITÉRIO)[COLUMNA].FUNÇÃO()
df.groupby("Bandeira")["Valor de Venda"].mean()
df.groupby("Produto")["Valor de Venda"].max()
df.groupby("Município")["CNPJ"].nunique() # postos distintos
df.groupby("Bandeira").size() # qtd linhas
df.groupby(["Município","Produto"])["Valor de Venda"].mean()
df.groupby("X", as_index=False)["v"].mean() # crit. vira coluna

# Filtro + groupby:
df[df["Produto"]=="GASOLINA"].groupby("Município")["Valor de
Venda"].mean()
```

Funções: mean, sum, median, min, max, std, size (linhas), count (não-nulos), nunique, first, last.

14. AGG (VÁRIAS ESTATÍSTICAS)

```
df.groupby("Bandeira")["Valor de
Venda"].agg(["mean","min","max","std"])

df.groupby("Bandeira").agg({
    "Valor de Venda": "mean",
    "CNPJ": "nunique",
})

df.groupby("Bandeira").agg({
    "Valor de Venda": ["mean","min","max"],
    "CNPJ": "nunique",
})
```

15. RANKING, TOP N, IDXMAX

```
df.groupby("B")["v"].mean().sort_values(ascending=False)
df.groupby("Município")
["v"].mean().sort_values(ascending=False).head(5)

# RÓTULO do maior/menor (não o valor):
df.groupby("Município")["v"].mean().idxmax() # nome
df.groupby("Município")["v"].mean().max() # valor
```

18. CASO REAL: POSTOS + DISTRIB (MARGEM)

1. Cria Mes em postos (formato YYYY-MM):
postos["Mes"] = (postos["Data da Coleta"].str[6:10] + "-"
+ postos["Data da Coleta"].str[3:5])

2. Junta pela chave TRIPLA (sem isso, cartesiano):
juntos = pd.merge(postos, distrib,
on=["Município","Produto","Mes"], how="inner")

3. Margem (revenda - atacado):
juntos["margem"] = juntos["Valor de Venda"] -
juntos["Preco_Distribuicao"]

4. Margem média por bandeira (ranking):
juntos.groupby("Bandeira")
["margem"].mean().sort_values(ascending=False)

19. CENÁRIOS TÍPICOS (COLE ADAPTANDO)

Postos distintos em SP capital
df[df["Município"]=="SAO PAULO"]["CNPJ"].nunique()

Top 5 cidades gasolina mais cara
(df[df["Produto"]=="GASOLINA"]
.groupby("Município")["Valor de Venda"].mean()
.sort_values(ascending=False).head(5))

Bandeira mais freq. em SP
df[df["Município"]=="SAO PAULO"]
["Bandeira"].value_counts().head(1)

Município com diesel mais caro (nome)
(df[df["Produto"]=="DIESEL S10"]
.groupby("Município")["Valor de Venda"].mean().idxmax())

Preço médio mês a mês de gasolina em SP
sp = df[(df["Produto"]=="GASOLINA")
& (df["Município"]=="SAO PAULO")].copy()
sp["Mes"] = sp["Data da Coleta"].str[6:10]+"-"+sp["Data da
Coleta"].str[3:5]
sp.groupby("Mes")["Valor de Venda"].mean()

Revenda contém "AUTO POSTO" E bandeira VIBRA
c1 = df["Revenda"].str.contains("AUTO POSTO")
c2 = df["Bandeira"]=="VIBRA"
df[c1 & c2]

Municípios com gasolina máxima > 7
g = df[df["Produto"]=="GASOLINA"]
m = g.groupby("Município")["Valor de Venda"].max()
m[m > 7]

Distribuição por produto (1 linha)
df.groupby("Produto")["Valor de Venda"].describe()

Qtd bandeiras por cidade (top 10)
df.groupby("Município")
["Bandeira"].nunique().sort_values(ascending=False).head(10)

Mês mais caro pra gasolina
g = df[df["Produto"]=="GASOLINA"].copy()
g["Mes"] = g["Data da Coleta"].str[6:10]+"-"+g["Data da
Coleta"].str[3:5]
g.groupby("Mes")["Valor de Venda"].mean().idxmax()

Margem média por município (top 10)
juntos.groupby("Município")
["margem"].mean().sort_values(ascending=False).head(10)

2 resultados na mesma célula
display(df["a"].mean())
display(df["b"].mean())

20. PLOT RÁPIDO
df.plot(x="data", y="qtd", kind="line", marker="o",
title="X", xlabel="data", ylabel="qtd")
kind: line, bar, barh, pie, hist, box
Múltiplas séries: y=["c1","c2"]
df.groupby("Mes")["v"].mean().plot(kind="bar")

21. ERROS TÍPICOS → CAUSA

Erro	Causa
KeyError: 'X'	nome de coluna errado, OU df virou Series
FileNotFoundError	caminho relativo errado, usa absoluto
ParserError: expected N, saw M	faltou sep=";";
'Series' has no 'upper'	esqueceu .str
'method' object does not support item assignment	usou [em vez de (
'float' object does not support item assignment	a = b = expr em cadeia
TypeError em aritmética	coluna é string, usa astype(float)
SettingWithCopyWarning	filtrou sem .copy()
DF gigante após merge	chave incompleta no on
Output <bound method>	esqueceu () ao chamar função
Coluna numérica vira object	faltou decima=","

Dica do prof: erro vermelho, lê a última linha do traceback. É onde o pandas diz o problema.

22. REGRAS-MÃE (DECORA ESTAS)

- = atribui, == compara. Erro é clássico.
- Estatística sempre em coluna: df["c"].mean(), não df.mean().
- Atualizar coluna: esquerda é df["c"], nunca só df.
- Booleano: & | ~, cada cond entre parênteses.
- loc fim inclusivo, iloc fim exclusivo.
- Métodos com (); atributos sem (shape, columns, dtypes).
- sort_values não persiste sozinho. Reatribui.
- Sempre .str antes de função de string.
- copy() ao criar subset que vai ser modificado.
- Chave de merge incompleta → cartesiano. Confere shape depois.
- Kernel restart zera tudo. Roda tudo em ordem (Restart and Run All).
- Salva com Cmd+S a cada exercício.

23. FLUXO DOS 100 MIN

- 1-5': lê TODAS as perguntas. Classifica mental: filtro? groupby? merge?
- 5-10': setup, confere shape e head. Se Valor virou string, converte.
- 10-50': faz as fáceis primeiro (value_counts, mean, filtro). Trava pontos.
- 50-80': combinadas (filtro+groupby, merge, encadeamento).
- 80-85': Restart and Run All pra confirmar consistência.
- 85-100': eClass abre. Representante sobe .ipynb + responde questionário. Confere os 3 nomes.

24. MINI-GLOSSÁRIO EXCEL → PANDAS

Excel	Pandas
=AVERAGE (A:A)	df["a"].mean()
=COUNTIF (A:A,x)	(df["a"]==x).sum()
Filtro coluna	df[df["c"]==x]
Tabela dinâmica	groupby ou pivot_table
PROCV / VLOOKUP	pd.merge
Ordenar	sort_values
Remover duplicatas	drop_duplicates
Distinct count	nunique()